

PROJECT HELIX

Helix

RISC-V CPU | 5-CPI Internal Architecture Specification

5-stage sequential project | IF / ID / EX / MEM / WB | CPI = 5

DOCUMENT ID	HELIX-5CPI-IAS-001
PROJECT	Helix (RISC-V CPU, 5-CPI)
REVISION	1.0
DATE	13 June 2026
STATUS	Released
ARCHITECTURE	ARIA-32 (ARM-inspired RISC ISA, 32-bit)
ISA VERSION	2.0
PIPELINE	5-Stage Sequential — IF / ID / EX / MEM / WB
CPI	5 (one instruction in the pipeline at a time)
HAZARDS	None — no structural, data, or control hazards
DATA WIDTH	32-bit
IMEM / DMEM	4 KB / 4 KB (1024 x 32-bit words each)
TARGET HDL	Verilog 2001 / SystemVerilog (Vivado 2025)

1 Architecture Overview

ARIA-32 5-Stage is a 32-bit RISC processor that executes the same ISA as the single-cycle ARIA-32 but uses a **5-stage sequential pipeline**. Unlike a conventional pipelined design, only **one instruction occupies the pipeline at a time** — each instruction passes through all five stages (IF, ID, EX, MEM, WB) before the next instruction is fetched. This gives a CPI of exactly 5 and eliminates all structural, data, and control hazards by design. The architecture serves as an educational bridge between single-cycle and full-hazard-managed pipelined designs.

Feature	Detail
ISA Class	Custom 32-bit RISC (ARM-inspired, same encoding as single-cycle ARIA-32)
Execution Model	5-stage sequential — one instruction in flight at a time
CPI	5 (exactly, always)
Hazards	None — eliminated by design (no concurrent instructions)
Instruction Width	Fixed 32 bits
Pipeline Stages	IF(0) -> ID(1) -> EX(2) -> MEM(3) -> WB(4) -> IF(0)
Stage Sequencer	3-bit register stage[2:0] cycles 0->1->2->3->4->0
Pipeline Registers	4 banks: IF/ID ID/EX EX/MEM MEM/WB
Register File	16 x 32-bit (R0-R15; R13=SP, R14=LR)
DMEM write gate	<code>dmem_we = exmem_mem_write & (stage == 3)</code> — fires only in MEM stage
RF write gate	<code>rf_we = memwb_reg_write & (stage == 4)</code> — fires only in WB stage
Interrupts	IRQ (0x18) + FIQ (0x1C) — evaluated at WB only; instruction always retires cleanly
Memory	Harvard: IMEM 4 KB (APB write-only) / DMEM 4 KB

1.1 Stage Summary

Stage	#	Active Sub-modules	Latches into	Action
IF — Instruction Fetch	0	<code>inst_mem.v</code>	IF/ID reg	pc -> IMEM async read -> capture inst[31:0] and pc
ID — Decode + RF Read	1	<code>control.v</code> , <code>reg_file.v</code>	ID/EX reg	Decode ifid_inst; read RF ports A and B; sign-extend imm16
EX — Execute	2	<code>alu.v</code> , <code>cond_check.v</code>	EX/MEM reg	ALU op; branch targets; condition eval; mem addr; flags
MEM — Memory Access	3	<code>data_mem.v</code>	MEM/WB reg	<code>dmem_we</code> gated: STR/PUSH write; async read for LDR/POP
WB — Write Back	4	<code>reg_file.v (wr)</code> , <code>cpu_top</code>	—	<code>rf_we</code> gated: write result; update pc, sp, sr; stage -> 0

2 Pipeline Block Diagram

The diagram below shows the five pipeline stages, the four inter-stage pipeline register banks, and the PC feedback path. Each stage executes for exactly one clock cycle. The pipeline register banks are latched on the rising edge of `clk` at the end of each stage. The stage counter `stage[2:0]` advances 0->1->2->3->4->0, gating DMEM writes to stage 3 and RF writes to stage 4.

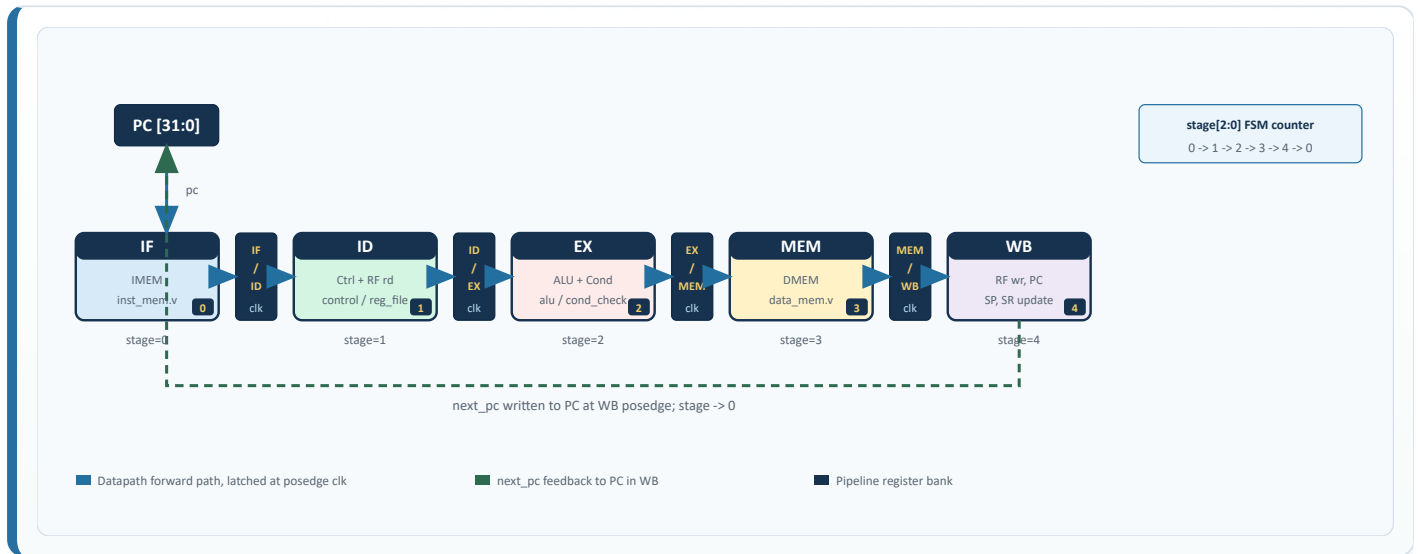


Figure 1 — ARIA-32 5-Stage Sequential Pipeline Datapath

2.1 Key Bus Signals

Signal	W	Description
pc	32	Updated at WB posedge to next_pc; drives IMEM in IF stage
imem_inst	32	Async read from IMEM[pc[11:2]]; captured into ifid_inst at IF posedge
stage[2:0]	3	Stage sequencer; cycles 0->1->2->3->4->0; gates dmem_we and rf_we
dmem_we	1	exmem_mem_write & (stage==3) – combinational, fires write at MEM posedge
rf_we	1	memwb_reg_write & (stage==4) – combinational, fires RF write at WB posedge
alu_result	32	ALU output in EX stage; becomes mem address or WB data
ex_cond_pass	1	Branch condition from cond_check; propagated to MEM/WB via pipeline regs
mem_rdata	32	Async DMEM read from exmem_mem_addr; captured into memwb_mem_rdata at MEM posedge
wb_data	32	WB mux: WB_ALU=alu_result, WB_MEM=mem_rdata, WB_PC4=pc+4 (BL)
next_pc	32	Mux: RETI/BX/BL/branch(+cond)/PC+4/FIQ_VEC/IRQ_VEC — written to pc in WB
sr[5:0]	6	Always current: [5:F][4:I][3:V][2:C][1:N][0:Z]; updated in WB
sp	32	Always current (no overlap); updated in WB from memwb_sp_next

3 Programmer's Model

3.1 General-Purpose Register File

Reg	Alias	Role	Reset
R0	—	General purpose — conventional return value / scratch	0x00000000
R1–R12	—	General purpose	0x00000000
R13	SP	Stack Pointer — maintained in cpu_top; mirrored in RF; used by PUSH/POP	0x0000FFFC
R14	LR	Link Register — receives PC+4 on BL; restored to PC by BX R14	0x00000000
R15	—	General purpose (PC is a separate register in cpu_top, not R15)	0x00000000

3.2 Status Register SR[5:0]

unused 31:6				F 5	I 4	V 3	C 2	N 1	Z 0
----------------	--	--	--	---------------	---------------	---------------	---------------	---------------	---------------

SR [bits 5:0 implemented; bits 31:6 unused]

Bit	Name	Description	Set by
0	Z — Zero	Set when ALU result == 0	ADD SUB MUL AND OR XOR NOT LSL LSR ASR INC DEC CMP TST
1	N — Negative	Set when ALU result[31] == 1	(same as Z)
2	C — Carry	Unsigned carry/borrow out; shift ejected bit	ADD SUB INC DEC LSL LSR ASR
3	V — Overflow	Signed overflow	ADD SUB INC DEC
4	I — IRQ en	1=IRQ enabled; cleared on IRQ entry; restored by RETI	Reset=1; IRQ entry; RETI
5	F — FIQ en	1=FIQ enabled; cleared on FIQ entry; restored by RETI	Reset=1; FIQ entry; RETI

4 Instruction Encoding

All instructions are exactly 32 bits. Bits [31:29] select the format; bits [28:24] select the operation. Encoding is identical to the single-cycle ARIA-32 — the same assembler and testbench programs work unchanged.

fmt[31:29]	Format	Typical Use
000	R-Type	Rd = Rs1 OP Rs2
001	I-Type	Rd = Rs1 OP #imm16
010	Load	Rd = mem[Rb + #off16]
011	Store	mem[Rb + #off16] = Rs
100	Branch	if(COND) PC += #imm25*4
101	Unary	INC / DEC / PUSH / POP
110	Jump	BL #off20 / BX Rd
111	Control	NOP / RETI

4.1 R-Type [fmt=000] — Rd = Rs1 OP Rs2

fmt[2:0] 31:29	funct[4:0] 28:24	Rd[3:0] 23:20	Rs1[3:0] 19:16	Rs2[3:0] 15:12	unused[11:0] 11:0
-------------------	---------------------	------------------	-------------------	-------------------	----------------------

R-Type [fmt=000] — Rd = Rs1 OP Rs2

4.2 I-Type [fmt=001] — Rd = Rs1 OP #imm16

fmt 31:29	funct 28:24	Rd 23:20	Rs1 19:16	imm16[15:0] 15:0
--------------	----------------	-------------	--------------	---------------------

I-Type [fmt=001] — Rd = Rs1 OP #imm16

4.3 Load [fmt=010] — Rd = mem[Rb + #off16]

fmt 31:29	funct 28:24	Rd 23:20	Rb 19:16	off16[15:0] 15:0
--------------	----------------	-------------	-------------	---------------------

Load [fmt=010] — Rd = mem[Rb + #off16]

4.4 Store [fmt=011] — mem[Rb + #off16] = Rs

fmt 31:29	funct 28:24	Rs 23:20	Rb 19:16	off16[15:0] 15:0
--------------	----------------	-------------	-------------	---------------------

Store [fmt=011] — mem[Rb + #off16] = Rs

4.5 Branch [fmt=100] — if(COND) PC += #imm25 x 4

fmt 31:29	COND[3:0] 28:25	imm25[24:0] 24:0
--------------	--------------------	---------------------

Branch [fmt=100] — if(COND) PC += #imm25 x 4

4.6 Unary [fmt=101] — INC/DEC/PUSH/POP Rd

fmt 31:29	funct 28:24	Rd 23:20	unused[19:0] 19:0
--------------	----------------	-------------	----------------------

Unary [fmt=101] — INC/DEC/PUSH/POP Rd

4.7 Jump [fmt=110] — BL #off20 / BX Rd

fmt 31:29	funct 28:24	Rd/0[3:0] 23:20	imm20[19:0] 19:0
--------------	----------------	--------------------	---------------------

Jump [fmt=110] — BL #off20 / BX Rd

4.8 Control [fmt=111] — NOP / RETI

fmt 31:29	funct 28:24	unused[23:0] 23:0
--------------	----------------	----------------------

Control [fmt=111] — NOP / RETI

5 Instruction Set Reference

5.1 R-Type Instructions [fmt=000]

Mnemonic	funct	Encoding	Operation	Flags
ADD Rd, Rs1, Rs2	0x00	{000,00000,Rd,Rs1,Rs2,0x000}	$Rd = Rs1 + Rs2$	Z N C V
SUB Rd, Rs1, Rs2	0x01	{000,00001,Rd,Rs1,Rs2,0x000}	$Rd = Rs1 - Rs2$	Z N C V
MUL Rd, Rs1, Rs2	0x02	{000,00010,Rd,Rs1,Rs2,0x000}	$Rd = Rs1 \times Rs2$ (lower 32 bits)	Z N
AND Rd, Rs1, Rs2	0x03	{000,00011,Rd,Rs1,Rs2,0x000}	$Rd = Rs1 \& Rs2$	Z N
OR Rd, Rs1, Rs2	0x04	{000,00100,Rd,Rs1,Rs2,0x000}	$Rd = Rs1 Rs2$	Z N
XOR Rd, Rs1, Rs2	0x05	{000,00101,Rd,Rs1,Rs2,0x000}	$Rd = Rs1 \wedge Rs2$	Z N
NOT Rd, Rs1	0x06	{000,00110,Rd,Rs1,-,0x000}	$Rd = \sim Rs1$	Z N
CMP Rs1, Rs2	0x07	{000,00111,-,Rs1,Rs2,0x000}	$Rs1 - Rs2$ (flags only, no write)	Z N C V
TST Rs1, Rs2	0x08	{000,01000,-,Rs1,Rs2,0x000}	$Rs1 \& Rs2$ (flags only, no write)	Z N
LSL Rd, Rs1, Rs2	0x09	{000,01001,Rd,Rs1,Rs2,0x000}	$Rd = Rs1 \ll Rs2[4:0]$	Z N C
LSR Rd, Rs1, Rs2	0x0A	{000,01010,Rd,Rs1,Rs2,0x000}	$Rd = Rs1 \gg Rs2[4:0]$ (logical)	Z N C
ASR Rd, Rs1, Rs2	0x0B	{000,01011,Rd,Rs1,Rs2,0x000}	$Rd = Rs1 \ggg Rs2[4:0]$ (arith)	Z N C
MOV Rd, Rs1	0x0C	{000,01100,Rd,Rs1,-,0x000}	$Rd = Rs1$ (no flag update)	—

5.2 I-Type Instructions [fmt=001]

The 16-bit immediate is sign-extended to 32 bits. Maximum positive MOVI value = 32767 (0x7FFF).

Mnemonic	funct	Operation	Flags
ADDI Rd, Rs1, #imm	0x00	$Rd = Rs1 + \text{SignExt}(\text{imm16})$	Z N C V
SUBI Rd, Rs1, #imm	0x01	$Rd = Rs1 - \text{SignExt}(\text{imm16})$	Z N C V
MOVI Rd, #imm	0x02	$Rd = \text{SignExt}(\text{imm16})$ (pass-B, no flags)	—
LSLI Rd, Rs1, #n	0x03	$Rd = Rs1 \ll n$ ($n = \text{imm16}[4:0]$)	Z N C
LSRI Rd, Rs1, #n	0x04	$Rd = Rs1 \gg n$ (logical)	Z N C
ASRI Rd, Rs1, #n	0x05	$Rd = Rs1 \ggg n$ (arithmetic)	Z N C

5.3 Memory Instructions [fmt=010 / 011]

Effective byte address = $Rb + \text{SignExt}(\text{off16})$. Word index = $\text{addr}[11:2]$.

Mnemonic	fmt	funct	Operation
LDR Rd, [Rb, #off]	fmt=010	0x00	$Rd = \text{mem}[Rb + \text{SignExt}(\text{off16})]$
STR Rs, [Rb, #off]	fmt=011	0x00	$\text{mem}[Rb + \text{SignExt}(\text{off16})] = Rs$

5.4 Branch Instructions [fmt=100]

Branch target = $PC + \text{SignExt}(\text{imm25}) \times 4$. Offset is from current instruction. Condition is evaluated in EX stage; result propagated to WB via pipeline regs.

Mnemonic	COND	Condition	Description
BEQ #off	0x1	Z==1	Branch if Equal
BNE #off	0x2	Z==0	Branch if Not Equal
BGT #off	0x3	~Z&N==V	Branch if Greater Than
BLT #off	0x4	N!=V	Branch if Less Than
BGE #off	0x5	N==V	Branch if Greater or Equal
BLE #off	0x6	Z N!=V	Branch if Less or Equal
BCS #off	0x7	C==1	Branch if Carry Set
BCC #off	0x8	C==0	Branch if Carry Clear
B #off	0xE	Always	Unconditional (COND_AL)
(NOP) #off	0x0	Never	Never-taken (COND_NV)

5.5 Unary Instructions [fmt=101]

Mnemonic	funct	Operation	Flags
INC Rd	0x00	Rd = Rd + 1	Z N C V
DEC Rd	0x01	Rd = Rd - 1	Z N C V
PUSH Rd	0x02	mem[SP-4] = Rd; SP -= 4 (full-descending)	—
POP Rd	0x03	Rd = mem[SP]; SP += 4	—

5.6 Jump Instructions [fmt=110]

Mnemonic	funct	Operation
BL #off20	0x00	LR(R14) = PC+4; PC = PC + SignExt(off20)*4
BX Rd	0x01	PC = Rd. Used for subroutine return (BX R14)

5.7 Control Instructions [fmt=111]

Mnemonic	funct	Operation
NOP	0x00	No operation; PC advances normally
RETI	0x01	Return from interrupt: PC = LR_irq/LR_fiq; SR restored from SPSR; mode -> SYS

The 4-bit COND field is evaluated by `cond_check.v` in the **EX stage** against the current SR flags. Because only one instruction is in the pipeline, SR is always up-to-date at EX time. The `ex_cond_pass` signal is latched into `exmem_cond_pass` and then `memwb_cond_pass`, and the branch decision is applied to PC in WB.

Code	Define	Condition	Expression	Mnemonic
0x0	COND_NV	Never	0 (constant false)	NOP / reserved
0x1	COND_EQ	Equal / Zero	Z	BEQ
0x2	COND_NE	Not Equal	~Z	BNE
0x3	COND_GT	Greater Than	~Z & (N==V)	BGT
0x4	COND_LT	Less Than	N != V	BLT
0x5	COND_GE	Greater or Equal	N == V	BGE
0x6	COND_LE	Less or Equal	Z (N != V)	BLE
0x7	COND_CS	Carry Set	C	BCS (unsigned >=)
0x8	COND_CC	Carry Clear	~C	BCC (unsigned <)
0x9–0xD	–	Reserved	–	(false)
0xE	COND_AL	Always	1 (constant true)	B (unconditional)
0xF	–	Reserved	–	(false)

7 Pipeline Architecture

This section describes the internal mechanics of the 5-stage sequential pipeline: the stage FSM, the contents of each pipeline register bank, a stage-by-stage operation table, and a worked timing example for two representative instructions.

7.1 Stage Sequencer (stage[2:0])

A 3-bit register `stage` in `cpu_top.v` acts as a 5-state FSM. It advances on every rising edge of `clk`. The `always @(posedge clk)` block is a `case(stage)` statement — each case arm implements exactly one pipeline stage.

stage	Name	Action on posedge clk
0	IF	Latch imem_inst and pc into IF/ID register; stage -> 1
1	ID	Decode ifid_inst; read reg_file; latch all control + data into ID/EX register; stage -> 2
2	EX	ALU executes; cond evaluated; targets computed; latch into EX/MEM register; stage -> 3
3	MEM	dmem_we fires combinationaly; async rdata captured into MEM/WB register; stage -> 4
4	WB	rf_we fires; wb_data written to reg_file; pc/sp/sr updated; stage -> 0

Why "stage==3" and "stage==4" gates are safe: The pipeline register contents (`exmem_mem_write`, `exmem_mem_addr`, etc.) remain stable for the entire clock cycle in stages 3 and 4. The combinational gates `dmem_we = exmem_mem_write & (stage==3)` and `rf_we = memwb_reg_write & (stage==4)` are asserted combinationaly for the full cycle and take effect at the rising clock edge. In all other stages the gates are de-asserted, preventing spurious writes even though the pipeline registers retain their previous values.

7.2 Pipeline Register Banks

Each bank is a set of registers latched at the posedge clk that ends the corresponding stage. Signal naming convention: `ifid_*` (latched at end of IF), `idex_*` (end of ID), `exmem_*` (end of EX), `memwb_*` (end of MEM).

IF/ID REGISTER BANK (LATCHED AT END OF IF, STAGE=0)

Signal	W	Source	Description
ifid_pc	32	pc	PC of the fetched instruction (byte address)
ifid_inst	32	imem_inst	Raw 32-bit instruction word read from IMEM

ID/EX REGISTER BANK (LATCHED AT END OF ID, STAGE=1)

Signal	W	Source	Description
idex_pc	32	ifid_pc	Pass-through PC
idex_rfa	32	d_rfa_data	RF port A read value (Rs1, or Rd for UNARY/BX)
idex_rfb	32	d_rfb_data	RF port B read value (Rs2, or Rd-field for STR)
idex_imm32	32	{{16{d_imm16[15]}} , d_imm16}	Sign-extended 16-bit immediate
idex_rd_addr	4	inst[23:20]	Destination register address
idex_fmt	3	inst[31:29]	Instruction format (needed for STR data mux in EX)
idex_br_cond	4	inst[28:25]	Branch condition code
idex_br_imm25	25	inst[24:0]	Branch word offset
idex_bl_imm20	20	inst[19:0]	BL word offset
idex_alu_op	4	control	Internal ALU opcode
idex_use_imm	1	control	Use idex_imm32 for ALU port B

Signal	W	Source	Description
idex_reg_write	1	control	Enable register file write in WB
idex_mem_read	1	control	Memory read (LDR, POP)
idex_mem_write	1	control	Memory write (STR, PUSH)
idex_wb_sel	2	control	WB mux: 00=ALU, 01=MEM, 10=PC+4
idex_branch / bl_op / bx_op / push_op / pop_op / reti_op / cmp_tst_op	1 each	control	Operation-type flags from decoder

EX/MEM REGISTER BANK (LATCHED AT END OF EX, STAGE=2)

Signal	W	Source	Description
exmem_alu_result	32	alu_result	ALU output value
exmem_mem_addr	32	ex_mem_addr	Effective memory address (ALU result, or SP+/-4 for stack)
exmem_mem_wdata	32	ex_mem_wdata	Write data (idex_rfb for STR; idex_rfa for PUSH)
exmem_pc4	32	idex_pc + 4	Return address for BL (written to LR in WB)
exmem_cond_pass	1	ex_cond_pass	Condition evaluator result — branch taken if 1
exmem_branch_target	32	ex_branch_target	PC + SignExt(imm25)*4
exmem_bl_target	32	ex_bl_target	PC + SignExt(imm20)*4
exmem_rfa	32	idex_rfa	Forwarded for BX Rd target
exmem_sp_next	32	ex_sp_next	New SP value (sp-4 for PUSH, sp+4 for POP)
exmem_sr_flags	4	{V,C,N,Z}	New flag values from ALU
exmem_update_flags	1	alu_flags_we & ~FMT_BRANCH	Enable SR flag update in WB
(+ control pass-throughs)	—	idex_*	reg_write, mem_read, mem_write, wb_sel, sp_we, branch, bl_op, bx_op, reti_op

MEM/WB REGISTER BANK (LATCHED AT END OF MEM, STAGE=3)

Contains the same fields as EX/MEM (all prefixed `memwb_`) plus the memory read data. The key addition:

Signal	W	Source	Description
memwb_mem_rdata	32	mem_rdata	Async DMEM read of exmem_mem_addr — captured at end of MEM stage

All other `memwb_*` signals are direct copies of the corresponding `exmem_*` fields latched at the posedge clk that ends stage 3.

7.3 Stage-by-Stage Operation Detail

Stage	Combinational activity during this cycle	Latched at posedge clk
0 – IF	IMEM async read of current pc; imem_inst immediately valid.	ifid_inst <= imem_inst; ifid_pc <= pc; stage <= 1
1 – ID	control decoder combinationaly decodes ifid_inst. RF async reads ports A and B (driven by inst fields). imm16 sign-extended.	All idex_* latched; stage <= 2
2 – EX	ALU operates on idex_rfa and (idex_rfb or idex_imm32). cond_check evaluates idex_br_cond against current SR. Branch targets, SP delta, mem address all computed combinationaly.	All exmem_* latched; stage <= 3
3 – MEM	dmem_we = exmem_mem_write & (stage==3) is HIGH this cycle. STR/PUSH: write fires at posedge. LDR/POP: mem_rdata valid combinationaly from exmem_mem_addr.	memwb_mem_rdata <= mem_rdata; all other memwb_* <= exmem_*; stage <= 4
4 – WB	rf_we = memwb_reg_write & (stage==4) is HIGH. WB mux selects ALU/MEM/PC4. RF write fires at posedge. pc <= next_pc; sp updated if PUSH/POP; sr[3:0] updated if flag-setting instruction. Interrupt checked here.	RF written; pc <= next_pc; stage <= 0

7.4 Execution Timing Example

Two instructions traced cycle-by-cycle. `ADD R3, R1, R2` at PC=0x00C0 followed by `LDR R0, [R5, #0]` at PC=0x00C4.

Clk	Stage	Instruction	Activity	Latched
1	0 - IF	ADD R3, R1, R2	IMEM[0x00C0] read async -> ifid_inst; ifid_pc=0x00C0; stage->1	ifid_inst, ifid_pc
2	1 - ID	ADD R3, R1, R2	Decode: ADD, R3<-R1+R2; RF reads R1->rfa, R2->rfb; stage->2	idex_rfa(R1), idex_rfb(R2), idex_alu_op(ADD), ...
3	2 - EX	ADD R3, R1, R2	ALU: R1+R2=result; flags {V,C,N,Z} computed; stage->3	exmem_alu_result, exmem_sr_flags, ...
4	3 - MEM	ADD R3, R1, R2	Not a mem op; dmem_we=0; results pass through; stage->4	memwb_alu_result, memwb_sr_flags, ...
5	4 - WB	ADD R3, R1, R2	rf_we=1: R3 written with result; SR[3:0] updated; pc<=0x00C4; stage->0	R3=R1+R2 in reg_file; pc=0x00C4
6	0 - IF	LDR R0, [R5, #0]	IMEM[0x00C4] read -> ifid_inst; stage->1	ifid_inst(LDR), ifid_pc=0x00C4
7	1 - ID	LDR R0, [R5, #0]	Decode: LDR, mem_read=1, wb_sel=WB_MEM; RF reads R5->rfa (base); imm32=0; stage->2	idex_rfa(R5), idex_imm32(0), idex_mem_read=1
8	2 - EX	LDR R0, [R5, #0]	ALU: R5+0=addr; ex_mem_addr=R5; stage->3	exmem_mem_addr=R5, exmem_mem_read=1, ...
9	3 - MEM	LDR R0, [R5, #0]	DMEM[R5] async read -> mem_rdata valid; dmem_we=0 (read, not write); stage->4	memwb_mem_rdata=DMEM[R5]
10	4 - WB	LDR R0, [R5, #0]	rf_we=1, wb_sel=WB_MEM: R0 written with memwb_mem_rdata; pc<=0x00C8; stage->0	R0=DMEM[R5] in reg_file; pc=0x00C8

Blue horizontal line separates instruction A (cycles 1-5) from instruction B (cycles 6-10).

8 Sub-Module Descriptions

8.1 cpu_top.v — Top-Level Integration (5-Stage)

The top module instantiates all sub-blocks and implements the stage-sequencing FSM. Sequential state (pc, sp, sr, stage, cpu_mode, lr_irq/fiq, spsr_irq/fiq, and all four pipeline register banks) is held in registers and updated on `posedge clk`. Combinational logic drives the sub-module inputs from the appropriate pipeline register bank for each stage. The APB bus runs on the independent `pclk`.

Critical design invariants:

- Because instructions never overlap, `sp` and `sr` are always the committed architectural values when read in EX and WB respectively.
- `dmem_we = exmem_mem_write & (stage==3)` — the gate ensures writes fire *only* at the MEM stage posedge and not spuriously in other stages where exmem registers retain stale values.
- `rf_we = memwb_reg_write & (stage==4)` — identical gate for the register file write.
- The halt/infinite-loop detection in testbenches must guard on `stage==4` before comparing PC, because PC changes only in WB. Checking every cycle gives 4 false positives per instruction.

Signal	Dir	W	Description
clk	in	1	CPU execution clock (drives stage FSM)
pclk	in	1	APB program-load clock (independent of clk)
rst_n	in	1	Active-low CPU reset: clears pc, sp, sr, stage, all pipeline registers
prst_n	in	1	Active-low APB reset: gates APB, does NOT clear IMEM
irq	in	1	Maskable IRQ — only accepted when stage==4 and SR[I]=1
fiq	in	1	Fast IRQ — only accepted when stage==4 and SR[F]=1
paddr / psel / penable / pwrite / pwrite	in	—	APB completer interface to inst_mem
prdata / pready / pslverr	out	—	APB responses from inst_mem

8.2 inst_mem.v — Instruction Memory (4 KB, APB Completer)

1024-word x 32-bit synchronous-write / asynchronous-read RAM. Loaded via APB with `rst_n=0`, `prst_n=1`. The CPU fetch stage reads combinatorially from the `cpu_addr` port (driven by `pc`) during stage 0 (IF). IMEM contents survive `rst_n` assertion.

8.3 control.v — Two-Level Instruction Decoder

Purely combinational. Decodes `ifid_inst` during stage 1 (ID). The outputs are all latched into the ID/EX register bank at the end of stage 1. Unchanged from single-cycle version.

8.4 reg_file.v — 16 x 32-bit Register File

Two asynchronous read ports; one synchronous write port. Reads are driven by `ifid_inst` fields during stage 1 (ID). The write port is driven by WB mux outputs and is gated by `rf_we` (stage==4). Because no two instructions execute simultaneously there is no read-after-write hazard — WB completes before the next IF begins.

8.5 alu.v — Arithmetic and Logic Unit

Fully combinational. Inputs driven by `idex_rfa` and `idex_rfb` or `idex_imm32` (selected by `idex_use_imm`) during stage 2 (EX). Produces `alu_result`, `alu_Z/N/C/V`, and `alu_flags_we`.

alu_op	Define	Operation	Flags	Used by
0x0	ALU_ADD	a + b	Z N C V	ADD ADDI INC

alu_op	Define	Operation	Flags	Used by
0x1	ALU_SUB	a - b	Z N C V	SUB SUBI DEC CMP
0x2	ALU_MUL	a x b (32-bit)	Z N	MUL
0x3	ALU_AND	a & b	Z N	AND TST
0x4	ALU_OR	a b	Z N	OR
0x5	ALU_XOR	a ^ b	Z N	XOR
0x6	ALU_NOT	~a	Z N	NOT
0x7	ALU_LSL	a << b[4:0]	Z N C	LSL LSLI
0x8	ALU_LSR	a >> b[4:0] (logical)	Z N C	LSR LSRI
0x9	ALU_ASR	\$signed(a) >>> b[4:0]	Z N C	ASR ASRI
0xA	ALU_MOV	a (pass-through)	—	MOV
0xB	ALU_MOVB	b (imm32)	—	MOVI
0xC	ALU_INC	a + 1	Z N C V	INC
0xD	ALU_DEC	a - 1	Z N C V	DEC

8.6 data_mem.v — Data Memory (4 KB)

1024-word x 32-bit synchronous-write / asynchronous-read RAM. The read address `exmem_mem_addr` is stable throughout stage 3 (MEM), so `mem_rdata` is valid combinationally and captured into `memwb_mem_rdata` at the posedge that ends stage 3. The write enable is combinationally gated to stage 3 only — the same posedge that latches the MEM/WB register also commits the STR or PUSH write to memory.

8.7 cond_check.v — Condition Code Evaluator

Purely combinational. Evaluates the 4-bit COND field (`idex_br_cond`) against `sr[Z/N/C/V]` during stage 2 (EX). Because SR is only updated in WB (stage 4) and instructions do not overlap, SR is guaranteed to contain the committed flags of the previous instruction at this point.

9 APB Programming Interface

Programs are loaded into IMEM via the AMBA APB bus. The CPU must be held in reset (`rst_n=0`) during loading. The APB clock `pclk` is independent of the CPU clock `clk`.

9.1 Load Sequence

Step	Signal State	Action
1	<code>rst_n=0</code> , <code>prst_n=1</code>	CPU halted at stage=0; APB enabled
2	APB SETUP phase	Set <code>paddr</code> , <code>psel=1</code> , <code>pwrite=1</code> , <code>pdata=instruction_word</code>
3	APB ACCESS phase	Assert <code>penable=1</code> ; <code>pready</code> goes high; word written to IMEM
4	Repeat per word	Increment <code>paddr</code> by 4 per instruction
5	<code>rst_n=1</code>	Release CPU reset; execution begins at PC=0x00000000, stage=0 (IF)

9.2 Memory Map Conventions

Byte Addr	Word	Convention
0x000	0	Boot vector: B AL #entry_offset (jump past vectors)
0x018	6	IRQ vector: RETI (or ISR entry)
0x01C	7	FIQ vector: RETI (or ISR entry)
0x080	32	Conventional program start (B AL #32 from boot)
0x100	64	Conventional DMEM data base for testbenches
0x200	128	Conventional DMEM result base for testbenches
0xFFC	1023	SP initial value (top of DMEM, full-descending stack)

10 Interrupt Architecture

ARIA-32 5-Stage supports two interrupt levels: FIQ (Fast, higher priority) and IRQ (Interrupt Request). A key design decision distinguishes the 5-stage version from the single-cycle: **interrupts are evaluated and accepted exclusively in stage 4 (WB)**. This guarantees that the instruction currently in the pipeline always retires cleanly — its register write-back, PC update, SP update, and flag update all complete before the interrupt is taken. There is therefore no partial instruction to undo on interrupt entry, and the return address is always well-defined.

Interrupt acceptance condition (stage 4 only):

```
fiq_taken = fiq & sr[F] & (cpu_mode==SYS) & (stage==4)
irq_taken = irq & sr[I] & (cpu_mode==SYS) & (stage==4) & ~fiq_taken
```

Parameter	IRQ	FIQ
Vector address	0x00000018	0x0000001C
Enable flag	SR[I] (bit 4)	SR[F] (bit 5)
Saved PC	lr_irq = normal_next_pc	lr_fiq = normal_next_pc
Saved SR	spsr_irq = SR	spsr_fiq = SR
On entry	SR[I]<-0; mode<-IRQ	SR[F]<-0; SR[I]<-0; mode<-FIQ
Return	RETI: PC<-lr_irq; SR<-spsr_irq; mode<-SYS	RETI: PC<-lr_fiq; SR<-spsr_fiq; mode<-SYS
Priority	Lower	Higher (evaluated first)
Accepted in stage	WB (stage=4) only	WB (stage=4) only
Accepted in mode	SYS only	SYS only
Interrupt latency	Max 5 cycles (worst: accepted just after IF of preceding instruction)	Same as IRQ

Saved return address: `normal_next_pc` (the address the non-interrupted flow would have gone to) is saved as the link register. For a non-branch instruction this equals `PC+4`. For a branch instruction that evaluated true, it equals the branch target. The RETI instruction therefore returns exactly to the correct next instruction in all cases.

Interrupt latency: Because the interrupt is only sampled at stage 4, the worst-case latency from IRQ/FIQ assertion to vector fetch is 5 clock cycles (when the interrupt arrives just as stage 0 begins for the preceding instruction). Best case is 1 cycle (interrupt arrives during stage 3 of the current instruction, sampled at stage 4).